Why late timesteps dominate the XOPD per-step loss

A flow-matching account of the `d_k` blow-up, artifact vs. signal, and the `xopd_dk_space` reweighting

Jayce-Ping

Abstract

Theory companion to the empirical observation in the July 12 per-timestep dominance progress note (last quarter of steps $\approx$ 87% of $\sum_k d_k$, final step $\approx$ 46% at `mean_tail`); the implemented remedy is `train.xopd_dk_space="x0"` (clean-latent loss, §5). Relevant code: `compute_per_step_kl` in `src/flow_factory/trainers/xopd/common.py`, called by `_precompute_d_per_timestep` / `_optimize_train_pass` in `src/flow_factory/trainers/xopd/trainer.py`.

This note defines four implemented per-step knobs (L_{xt}, L_v, L_{x_0}, $L_{x_0}^{\mathrm{norm}}$) plus the Gaussian transition KL L_{KL} (DanceOPD theory, §1.1) — formulas in the five-loss summary — their exact conversion on the FLUX.2-klein 28-step grid, and why the default L_{xt} (config `"xt"`) dominates late timesteps. Companion data: §3–§4.

Contents

Five per-step losses at a glance

At every discrete step k, student and teacher are evaluated on the *same* on-policy state x_t; only their velocity predictions v_s (student) and v_t (teacher) differ. Let $\sigma = t/1000$ be the FM noise fraction and $\Delta t = \sigma_{k+1}^{\text{sched}} - \sigma_k^{\text{sched}} < 0$ the schedule step. The five losses compared throughout this note are:

$$L_{xt} = \|\mu_s - \mu_t\|^2 \qquad \text{(default; \texttt{xopd\_dk\_space="xt"})}, \tag{1}$$

$$L_v = \|v_s - v_t\|^2 \qquad \text{(\texttt{xopd\_dk\_space="v"})}, \tag{2}$$

$$L_{x_0} = \|z_{0,s} - z_{0,t}\|^2 \qquad \text{(\texttt{xopd\_dk\_space="x0"})}, \tag{3}$$

$$L_{x_0}^{\text{norm}} = \frac{\|z_{0,s} - z_{0,t}\|^2}{\operatorname{sg}\big(\text{mean}\,|z_{0,s} - z_{0,t}|\big) + \varepsilon} = \frac{L_{x_0}}{\operatorname{sg}\big(\text{mean}\,|z_{0,s} - z_{0,t}|\big) + \varepsilon} \qquad \text{(\texttt{xopd\_dk\_space="x0\_norm"})}, \tag{4}$$

$$L_{\text{KL}} = \text{KL}\big(p_m(\cdot \mid x_t) \,\|\, p_\theta(\cdot \mid x_t)\big) = \frac{\Delta t^2}{2\sigma^2} L_v = \frac{L_{xt}}{2\sigma^2} \qquad \text{(DanceOPD transition KL)}. \tag{5}$$

Definitions via the velocity bridge v_s, v_t. The transition means and clean latents are *deterministic functions* of the shared state x_t and each model's velocity:

$$\mu_s = x_t + v_s\,\Delta t, \qquad \mu_t = x_t + v_t\,\Delta t \qquad \text{(ODE one-step means)}, \tag{6}$$

$$z_{0,s} = x_t - \sigma\,v_s, \qquad z_{0,t} = x_t - \sigma\,v_t \qquad \text{(recovered clean latents)}. \tag{7}$$

Because x_t is shared, it cancels in every difference, so $v_s - v_t$ is the single bridge:

$$\mu_s - \mu_t = \Delta t\,(v_s - v_t), \tag{8}$$

$$z_{0,s} - z_{0,t} = -\sigma\,(v_s - v_t). \tag{9}$$

Squaring both sides immediately yields the ODE identities $L_{xt} = \Delta t^2 L_v$, $L_{x_0} = \sigma^2 L_v$, and hence $L_{\text{KL}} = L_{xt}/(2\sigma^2) = \Delta t^2 L_v/(2\sigma^2)$ (§1). The self-normalized variant $L_{x_0}^{\text{norm}}$ (4) divides L_{x_0} by a *detached* per-sample mean-abs scale (sg = stop-gradient; $\varepsilon > 0$ floors the denominator), so it is scale-invariant and *not* a fixed schedule reweighting of L_v (§10). The first four losses are implemented knobs in `compute_per_step_kl`; L_{KL} is the theoretical same-variance Gaussian transition KL (§1.1).

1 Three per-step MSE losses and their exact conversion

XOPD compares student and teacher on the *same* on-policy rollout state x_t at step index k; only their predictions differ. Under rectified-flow interpolation ($x_t = (1-\sigma)z_0 + \sigma\varepsilon$, $\sigma = t/1000$) and ODE Euler stepping ($\mu = x_t + v\,\Delta t$, $\Delta t = \sigma_{k+1}^{\text{sched}} - \sigma_k^{\text{sched}} < 0$), there are three equivalent fixed MSE targets plus one adaptive self-normalized variant (config `xopd_dk_space`, default `"xt"`; definitions (1)–(4)):

knob	loss	formula	note
`xt`	L_{xt}	$\|\mu_s - \mu_t\|^2$ (1)	transition mean / next latent (default)
`v`	L_v	$\|v_s - v_t\|^2$ (2)	raw velocity MSE; ODE-only
`x0`	L_{x_0}	$\|z_{0,s} - z_{0,t}\|^2$ (3)	clean-latent MSE; ODE-only
`x0_norm`	$L_{x_0}^{\text{norm}}$	$L_{x_0}/(\operatorname{sg}(\text{mean}\|z_{0,s} - z_{0,t}\|) + \varepsilon)$ (4)	DiffusionNFT/DMD self-norm; ODE-only

Ratio. $\mathrm{MSE}(v) : \mathrm{MSE}(xt) : \mathrm{MSE}(x0) = 1 : \Delta t^2 : \sigma^2$.

Here μ is the one-step transition mean ($x_{t+\Delta t}$ under ODE), v the model velocity (`noise_pred`), and $z_0 = x_t - \sigma v$ the analytically recovered clean latent. Because x_t cancels:

$$\mu_s - \mu_t = \Delta t\,(v_s - v_t), \tag{10}$$

$$z_{0,s} - z_{0,t} = -\sigma\,(v_s - v_t), \tag{11}$$

$$\Rightarrow \quad L_{xt} = \Delta t^2\, L_v, \tag{12}$$

$$L_{x_0} = \sigma^2\, L_v, \tag{13}$$

$$L_{x_0} = \left(\tfrac{\sigma}{\Delta t}\right)^2 L_{xt}, \tag{14}$$

$$L_v = \left(\tfrac{\Delta t}{\sigma}\right)^2 L_{x_0}. \tag{15}$$

All four identities are *exact* on the discrete ODE grid (no approximation). The conversion factors depend *only* on $(\sigma_k, \Delta t_k)$ at step k; they do not require equal errors across steps — they convert *the same* underlying $(v_s - v_t)$ disagreement between spaces.

Conversion matrix (multiply to go from column loss to row loss).

	$\to L_{xt}$	$\to L_v$	$\to L_{x_0}$
from L_{xt}	1	Δt^{-2}	$(\sigma/\Delta t)^2$
from L_v	Δt^2	1	σ^2
from L_{x_0}	$(\Delta t/\sigma)^2$	σ^{-2}	1

What each loss implicitly weights (if you sum $\sum_k L_{\{\cdot\},k}$ with coefficient 1). Fix the *underlying* velocity disagreement $L_{v,k}$ at step k:

- L_{xt} **(default):** step k contributes $\Delta t_k^2 L_{v,k}$ — late steps have larger $|\Delta t_k|$ and dominate (§3, col. "$\%L_{xt}$").
- L_v (`xopd_dk_space="v"`)**:** step k contributes $L_{v,k}$ directly — the native FM velocity target; under the fixed 28-step grid the per-step Δt_k vary only modestly so this is close to uniform (§4).
- L_{x_0} (`xopd_dk_space="x0"`)**:** step k contributes $\sigma_k^2 L_{v,k}$ — early high-noise steps ($\sigma \approx 1$) dominate (col. "pct L_{x_0}").

Alternatively, fix $L_{x_0,k}$ (equal clean-latent disagreement): $L_{xt,k} = (\Delta t_k/\sigma_k)^2 L_{x_0,k}$, which is the "$(\Delta t/\sigma)^2$ blow-up" narrative (§6).

1.1 Fourth form: Gaussian transition KL (DanceOPD / DiffusionOPD theory)

The three knobs above are *unweighted* per-step MSE targets. A separate, widely used objective is the **same-variance Gaussian transition KL** between teacher and student one-step distributions (definition (5)). Under a local model $p(\cdot \mid x_t) = \mathcal{N}(\mu, \sigma_t^2 I)$ with shared covariance σ_t^2 (DanceOPD Appendix 7.1; DiffusionOPD pathwise term), and $\mu = x_t + v\,\Delta t$:

$$\boxed{L_{\mathrm{KL},k} = \mathrm{KL}\big(p_m(\cdot \mid x_t)\,\big\|\,p_\theta(\cdot \mid x_t)\big) = \frac{\Delta t_k^2}{2\sigma_k^2}\,\|v_s - v_t\|^2 = \frac{L_{xt,k}}{2\sigma_k^2}.} \tag{16}$$

This is **not** equal to any of $\{L_{xt}, L_v, L_{x_0}\}$ with unit coefficient; it is a *fourth* reweighting of the same underlying velocity gap. Exact conversions (same $(v_s - v_t)$ at step k):

	$\to L_{xt}$	$\to L_v$	$\to L_{x_0}$	$\to L_{\mathrm{KL}}$
from L_{KL}	$2\sigma^2$	$2\sigma^2/\Delta t^2$	$2\sigma^4/\Delta t^2$	1
from L_{xt}	1	Δt^{-2}	$(\sigma/\Delta t)^2$	$(2\sigma^2)^{-1}$
from L_v	Δt^2	1	σ^2	$\Delta t^2/(2\sigma^2)$
from L_{x_0}	$(\Delta t/\sigma)^2$	σ^{-2}	1	$\Delta t^2/(2\sigma^4)$

Implicit weight on L_v. Summing $\sum_k L_{\mathrm{KL},k}$ with coefficient 1 applies $w_k^{\mathrm{KL}} = \Delta t_k^2/(2\sigma_k^2)$ to the underlying $L_{v,k}$ — *between* the L_{xt} weight (Δt_k^2) and the L_{x_0} weight (σ_k^2): early high-noise steps are down-weighted by $1/\sigma_k^2$ relative to L_{xt}, but late steps with small σ_k still *amplify* $1/\sigma_k^2$ enough that L_{KL} is **more** late-heavy than L_{xt} under flat L_v (Figure 1: **77**% vs. **41**% on `ti=27`).

DanceOPD vs. our `normalize_d_k`.

- **DanceOPD training default** is *plain* $L_v = \|v_s - v_t\|^2$ at a single semantic-side low-t query — (16) is the *theoretical* KL identity; their ablations report plain MSE beats KL-$\bar{\sigma}^2$ / Min-SNR reweightings (`docs/mof/danceopd_comparison.md`).
- **Our** `normalize_d_k=true` (only in `space="xt"`) divides transition mean MSE by $2\bar{\sigma}_k^2 = 2\,\mathrm{std\_dev}_t^2\,|\Delta t_k|$ (SDE Appendix B $\bar{\sigma}$), *not* by $2\sigma_k^2$ with FM fraction $\sigma_k = t_k/1000$. Under ODE ($\mathrm{std\_dev}_t \approx 0$) it falls back to plain L_{xt}. So (16) is **conceptually related** to normalized transition KL but **not implemented** as a separate knob; it differs from L_{KL} unless one identifies $\bar{\sigma}$ with σ_k and uses SDE dynamics.
- **Ratio to L_{xt}:** $L_{\mathrm{KL},k}/L_{xt,k} = 1/(2\sigma_k^2)$. On the FLUX.2-klein grid this *increases* late-step share vs. plain L_{xt} (77% vs. 41% on `ti=27` under flat L_v), because $\sigma_k \to 0$ faster than Δt_k^2 shrinks toward the end.

2 The key identity (legacy shorthand)

With `normalize_d_k=false`, the logged `train/d_k/{ti}` is L_{xt} (not L_v). Combining (12)–(15):

$$L_{xt} = \Delta t^2 L_v = \left(\frac{\Delta t}{\sigma}\right)^2 L_{x_0}.$$

So the default loss is clean-latent disagreement multiplied by the implicit per-step weight $(\Delta t/\sigma)^2$ (§1). The x-space knob sets L_{x_0} directly and removes that factor (§5).

3 Concrete schedule: official FLUX.2-klein @ 28 steps

We reproduce the *exact* denoising grid used in XOPD training (`num_inference_steps=28`, 512×512) by loading the default scheduler from the official HuggingFace repo and calling the same `retrieve_timesteps` path as `Flux2KleinPipeline` (shifted flow-matching sigmas with `image_seq_len=1024`, $\mu = 1.859$):

```
FlowMatchEulerDiscreteScheduler.from_pretrained(
    "black-forest-labs/FLUX.2-klein-base-4B", subfolder="scheduler")
# sigmas = linspace(1, 1/28, 28); mu = compute_empirical_mu(1024, 28)
```

Repro script: `.scratch/flux2_klein_scheduler_timesteps.py` (uv venv `/Users/bowenping/.venvs/flux2-klein-scheduler`, `diffusers==0.39.0`).

Notation at step index `ti`. `ti=0` is the noisiest state; `ti=27` is the last denoising step before $\sigma = 0$. The scheduler returns a *timestep* $t_k \in [0, 1000]$ and a *shifted flow sigma* $\sigma_k^{\mathrm{sched}} \in [0, 1]$. XOPD's clean-latent recovery uses the repo's FM fraction $\sigma_k = t_k/1000$ (`_noise_fraction`, `flow_match_sigma`). The ODE step size is $\Delta t_k = \sigma_{k+1}^{\mathrm{sched}} - \sigma_k^{\mathrm{sched}} < 0$ (from `scheduler.step`). On this grid $\sigma_k \approx \sigma_k^{\mathrm{sched}}$ to machine precision.

Per-step conversion factors on the 28-step grid. Table 1 lists representative steps; Table 2 (auto-generated, `docs/xopd/core/shared/flux2_klein_28step_loss_scales.tex`) gives all 28 rows. Repro: `.scratch/flux2_klein_scheduler_timesteps.py`.

`ti`	t	σ	Δt	$\frac{L_{xt}}{L_v} = \Delta t^2$	$\frac{L_{x0}}{L_v} = \sigma^2$	$\frac{L_{x0}}{L_{xt}}$	pct L_{xt}	pct L_{x_0}
0	1000.0	1.0000	−0.00574	$3.3{\times}10^{-5}$	1.000	$3.0{\times}10^{4}$	0.04%	5.28%
7	950.6	0.9506	−0.00929	$8.6{\times}10^{-5}$	0.904	$1.0{\times}10^{4}$	0.10%	4.77%
13	881.0	0.8810	−0.01584	$2.5{\times}10^{-4}$	0.776	$3.1{\times}10^{3}$	0.28%	4.10%
21	681.5	0.6815	−0.04505	$2.0{\times}10^{-3}$	0.464	$2.3{\times}10^{2}$	2.27%	2.45%
24	516.8	0.5168	−0.08175	$6.7{\times}10^{-3}$	0.267	40.0	7.47%	1.41%
25	435.1	0.4351	−0.10456	$1.1{\times}10^{-2}$	0.189	17.3	12.21%	1.00%
26	330.5	0.3305	−0.13847	$1.9{\times}10^{-2}$	0.109	5.70	21.42%	0.58%
27	192.1	0.1921	−0.19206	$3.7{\times}10^{-2}$	0.037	1.00	41.21%	0.19%

Table 1: FLUX.2-klein-base-4B, `num_inference_steps=28`, 512px. Columns 5–7: exact factors from (12)–(14). "%L_{xt}" ("%L_{x_0}"): share of $\sum_k L_{xt,k}$ (resp. $\sum_k L_{x0,k}$) if $L_{v,k} \equiv$ const across steps.

Reading the numbers.

- **At `ti=27`, all three losses coincide in scale.** $|\Delta t_{27}| \approx \sigma_{27}$, so $\Delta t_{27}^2 = \sigma_{27}^2 \approx 0.037$ and $L_{xt} = L_{x_0} = L_v$ for the *same* velocity gap.
- **Default L_{xt} dominance ≈ flat L_v.** If $\|v_s - v_t\|^2$ were identical at every step, $\sum_k L_{xt,k}$ puts **41**% on `ti=27` (col. "%L_{xt}"). The mixed XOPD run observes **46**% on `ti=27` at `mean_tail` — the logged curve matches *roughly constant velocity error* amplified only by Δt_k^2, not the stronger $(\Delta t_k/\sigma_k)^2$ factor (which would give **77**% if $L_{x0,k}$ were constant).
- **L_{x_0} up-weights early steps.** With flat L_v, high-noise steps each take $\sim 5\%$ of $\sum L_{x_0}$ while `ti=27` takes only **0.19**%. Setting `xopd_dk_space="x0"` switches to this allocation.
- **Cross-step ratios (same L_v).** $L_{xt,27}/L_{xt,13} = (\Delta t_{27}/\Delta t_{13})^2 \approx$ **148**×; $L_{x0,0}/L_{x0,27} = \sigma_0^2/\sigma_{27}^2 \approx$ **27**×.
- **"Late step" $\neq \sigma \to 0$.** The last rollout step has $t = 192$ ($\sigma = 0.19$), not $t \approx 0$. Dominance comes from the discrete grid's large $|\Delta t_k|$ at the end.

Implicit per-step shares (flat L_v). Figure 1 plots the normalized share of $\sum_k L_{\{\cdot\},k}$ at each `ti` when the underlying velocity gap is identical at every step ($L_{v,k} \equiv$ *const*). Blue/red/green: the three implemented MSE knobs (L_{xt}, L_v, L_{x_0}); orange: the Gaussian transition KL $L_{\mathrm{KL}} = \Delta t^2 L_v/(2\sigma^2)$ (§1.1). Its weight $\Delta t_k^2/(2\sigma_k^2)$ is *more* late-heavy than L_{xt} alone under flat L_v.

4 Empirical distribution: mixed run, converted from measured L_{xt}

We pull `train/d_k/{ti}` from two W&B runs in project `Flow-Factory-XOPD`, both with `xopd_dk_space=xt` and `normalize_d_k=false`: `4cnkluwk` (mixed geneval+OCR 1:1) and

ti	t	σ	Δt	$\frac{L_{xt}}{L_v}$	$\frac{L_{x0}}{L_v}$	$\frac{L_{x0}}{L_{xt}}$	pct L_{xt}	pct L_{x0}
0	1000.0	1.0000	-0.00574	3.3e-05	1.000	30380.38	0.04%	5.28%
1	994.3	0.9943	-0.00611	3.7e-05	9.89e-01	26518.80	0.04%	5.22%
2	988.2	0.9882	-0.00651	4.2e-05	9.76e-01	23039.00	0.05%	5.15%
3	981.6	0.9816	-0.00696	4.8e-05	9.64e-01	19912.21	0.05%	5.09%
4	974.7	0.9747	-0.00745	5.6e-05	9.50e-01	17114.06	0.06%	5.01%
5	967.2	0.9672	-0.00800	6.4e-05	9.36e-01	14621.44	0.07%	4.94%
6	959.2	0.9592	-0.00861	7.4e-05	9.20e-01	12410.80	0.08%	4.86%
7	950.6	0.9506	-0.00929	8.6e-05	9.04e-01	10460.53	0.10%	4.77%
8	941.3	0.9413	-0.01006	1.0e-04	8.86e-01	8748.98	0.11%	4.68%
9	931.3	0.9313	-0.01093	1.2e-04	8.67e-01	7256.11	0.13%	4.58%
10	920.3	0.9203	-0.01192	1.4e-04	8.47e-01	5962.25	0.16%	4.47%
11	908.4	0.9084	-0.01305	1.7e-04	8.25e-01	4849.23	0.19%	4.35%
12	895.4	0.8954	-0.01434	2.1e-04	8.02e-01	3899.25	0.23%	4.23%
13	881.0	0.8810	-0.01584	2.5e-04	7.76e-01	3095.63	0.28%	4.10%
14	865.2	0.8652	-0.01758	3.1e-04	7.49e-01	2422.59	0.35%	3.95%
15	847.6	0.8476	-0.01963	3.9e-04	7.18e-01	1865.26	0.43%	3.79%
16	828.0	0.8280	-0.02205	4.9e-04	6.86e-01	1409.55	0.54%	3.62%
17	805.9	0.8059	-0.02496	6.2e-04	6.50e-01	1042.44	0.70%	3.43%
18	781.0	0.7810	-0.02849	8.1e-04	6.10e-01	751.66	0.91%	3.22%
19	752.5	0.7525	-0.03281	1.08e-03	5.66e-01	525.93	1.20%	2.99%
20	719.7	0.7197	-0.03821	1.46e-03	5.18e-01	354.83	1.63%	2.73%
21	681.5	0.6815	-0.04505	2.03e-03	4.64e-01	228.85	2.27%	2.45%
22	636.4	0.6364	-0.05391	2.91e-03	4.05e-01	139.37	3.25%	2.14%
23	582.5	0.5825	-0.06567	4.31e-03	3.39e-01	78.683	4.82%	1.79%
24	516.8	0.5168	-0.08175	6.68e-03	2.67e-01	39.970	7.47%	1.41%
25	435.1	0.4351	-0.10456	1.09e-02	1.89e-01	17.315	12.21%	1.00%
26	330.5	0.3305	-0.13847	1.92e-02	1.09e-01	5.698	21.42%	0.58%
27	192.1	0.1921	-0.19206	3.69e-02	3.69e-02	1.000	41.21%	0.19%

Table 2: Three per-step MSE losses on the FLUX.2-klein 28-step grid (512×512). Columns 5–7 are exact conversion factors at step k (on-policy, shared x_t). Last two columns: normalized share of $\sum_k L_{xt}$ (resp. $\sum_k L_{x0}$) if $\|v_s - v_t\|^2$ were identical at every step. Generated from `black-forest-labs/FLUX.2-klein-base-4B` default scheduler.

`51ueyo7w` (geneval_enhanced specialist). Under ODE we convert each step to the *hypothetical* L_v and L_{x0} that would have been logged with the same underlying velocity gap:

$$L_{v,k} = \frac{L_{xt,k}}{\Delta t_k^2}, \qquad L_{x0,k} = \Big(\frac{\sigma_k}{\Delta t_k}\Big)^2 L_{xt,k}, \qquad L_{\mathrm{KL},k} = \frac{L_{xt,k}}{2\sigma_k^2}.$$

Windows. *initial* = mean of the first 4 logged optimizer steps (≈ first 4 epochs); *mean_tail* = mean of the last 30% of logged steps. Figures are generated by `.scratch/plot_per_timestep_loss_dominance.py` (repro: pull W&B, then plot).

Table 3 summarizes the mixed run `mean_tail` aggregates; values match Figure 2 (top-right panel).

aggregate	L_{xt} **(meas.)**	L_v **(conv.)**	L_{x0} **(conv.)**
top-1 (`ti=27`)	45.6%	0.5%	0.02%
top-7 (`ti=21--27`, last 1/4)	87.3%	3.4%	0.9%
high-noise half (`ti=0--13`)	6.9%	85.9%	92.1%
low-noise half (`ti=14--27`)	93.1%	14.1%	7.9%

Table 3: Share of $\sum_k L_{\{\cdot\},k}$ (`mean_tail`, mixed run `4cnkluwk`, 250 logged steps) in each loss space. L_{xt} is measured; L_v and L_{x0} are ODE-exact conversions. Geneval_enh specialist initial window: `ti=27` **42**%, `ti=21--27` **79**% (`51ueyo7w`).

Reading.

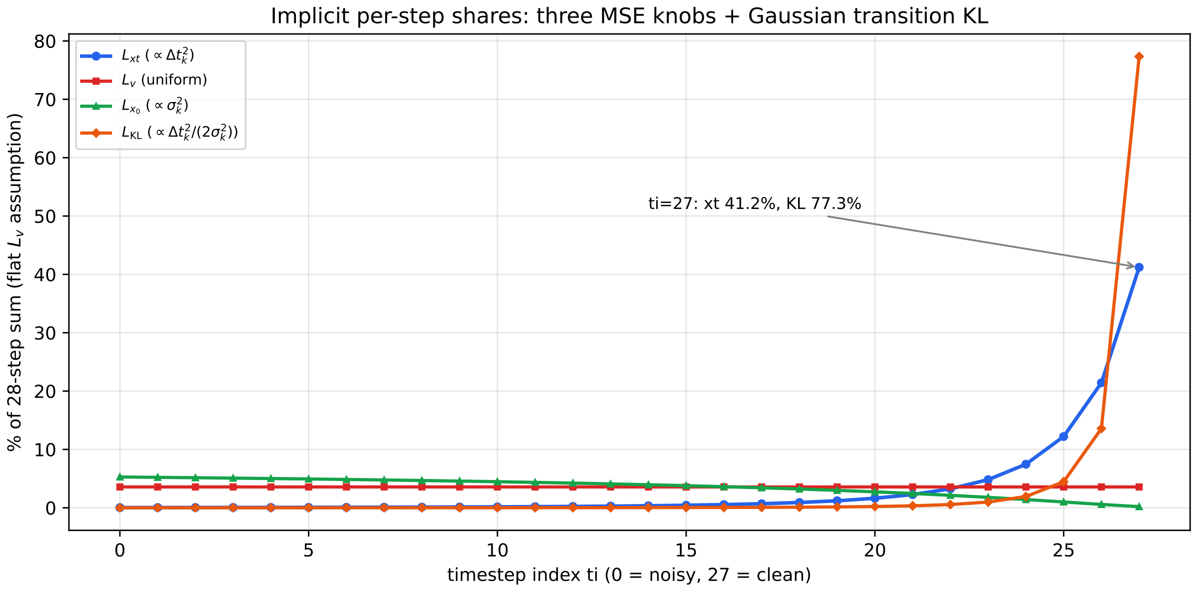

Figure 1: Theoretical per-`ti` shares under flat-L_v. Four curves: L_{xt}, L_v, L_{x_0}, and DanceOPD/DiffusionOPD transition KL $L_{\mathrm{KL}} = \Delta t^2/(2\sigma^2) \cdot L_v$. L_{KL} (orange) is **more** late-concentrated than L_{xt} (77% vs. 41% on `ti=27`) because $1/\sigma_k^2$ diverges as $\sigma_k \to 0$.

- **Late dominance is specific to L_{xt}.** Mixed `mean_tail` puts **46**% on `ti=27` and **87**% on the last quarter for L_{xt}; converted L_v and L_{x_0} put **86**%/**92**% on the high-noise half (`ti=0--13`). The logged curve is dominated by Δt_k^2, while the underlying velocity gap is larger at high noise (Figure 2).
- **Initial ≈ theoretical flat-L_v prediction.** Geneval_enh at epoch 0–3 already shows **42**% on `ti=27` for L_{xt}, close to the **41**% from Figure 1 — the dominance is largely a metric artifact, not something training must "learn" first.
- **L_v is nearly flat in absolute scale** after conversion (only **0.5**% of the sum on `ti=27`), consistent with Cause A (§6.1).
- **Ablation implication.** `xopd_dk_space="v"` removes the Δt_k^2 rescaling and should redistribute gradient toward high-noise steps; `"x0"` applies σ_k^2 and amplifies that further.

4.1 Training dynamics: does L_{xt} late dominance change over epochs?

Figure 4 tracks the L_{xt} and converted L_{KL} shares at `ti=27`, the last quarter (`ti=21--27`), and the high-noise half (`ti=0--13`, L_{xt} only) across logged optimizer steps (one step ≈ one epoch under XOPD). Both runs start with high late-step L_{xt} share (∼40% on `ti=27`) while L_{KL} is higher still (∼70–80%); both remain stable as training progresses. Bottom panel: gap $L_{\mathrm{KL}} - L_{xt}$ at `ti=27` (percentage points), showing the extra late weight from $1/(2\sigma_k^2)$.

5 Implementation: the `xopd_dk_space` knob

Knob. `train.xopd_dk_space` ∈ {`v`, `xt`, `x0`} (default `xt`). See `compute_per_step_kl` in `xopd/common.py`; `v` and `x0` require ODE (`XOPDTrainer.__init__` guard).

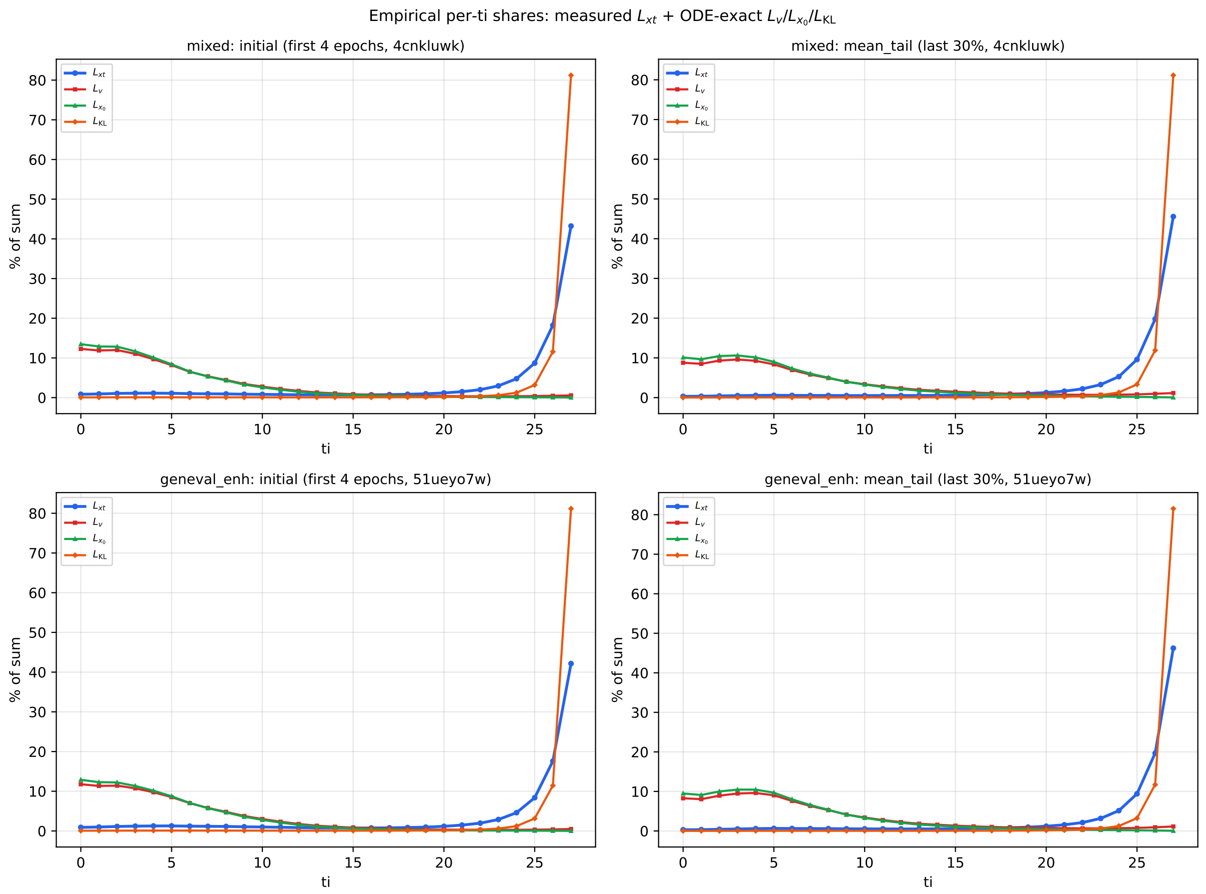

Figure 2: Empirical per-`ti` shares in each loss space. Top row: mixed run (`4cnkluwk`); bottom row: geneval_enh specialist (`51ueyo7w`). Left: initial window; right: `mean_tail`. Orange: converted $L_{\mathrm{KL}} = L_{xt}/(2\sigma_k^2)$ (§1.1). Late dominance is strongest in L_{KL}, absent in L_v/L_{x_0}.

knob	**per-step loss**	**relation to L_v**
`xt`	$L_{xt} = \|\mu_s - \mu_t\|^2$	$L_{xt} = \Delta t^2 L_v$
`v`	$L_v = \|v_s - v_t\|^2$	identity
`x0`	$L_{x_0} = \|z_{0,s} - z_{0,t}\|^2$	$L_{x_0} = \sigma^2 L_v$

Since x_t cancels, $L_{x_0} = (\sigma/\Delta t)^2 L_{xt}$ ((14)). `normalize_d_k` applies only to `xt`; `v`/`x0` ignore it.

No extra cost. `x0` recovers z_0 analytically from μ; `v` uses $(\mu_s - \mu_t)/\Delta t$. **Logging.** `train/d_k/{ti}` values are in the selected space; compare *shapes* across spaces, not raw magnitudes (§4).

6 Two causes of the dominance: separate the artifact from the signal

6.1 Cause A — Δt_k^2 amplification in L_{xt} (a parameterization artifact)

The default loss is $L_{xt} = \Delta t^2 L_v$ ((12)), not velocity MSE. Summing $\sum_k L_{xt,k}$ with unit weight therefore applies a per-step factor Δt_k^2 to the underlying velocity gap. Late rollout steps have larger $|\Delta t_k|$ (Table 1), so L_{xt} **over-represents late steps even if L_v were flat**.

Quantitative check on the FLUX.2-klein 28-step grid (§3):

- If $L_{v,k} \equiv$ const: $\sum_k L_{xt,k}$ puts **41**% on `ti=27` (col. "%L_{xt}"). The mixed XOPD run observes

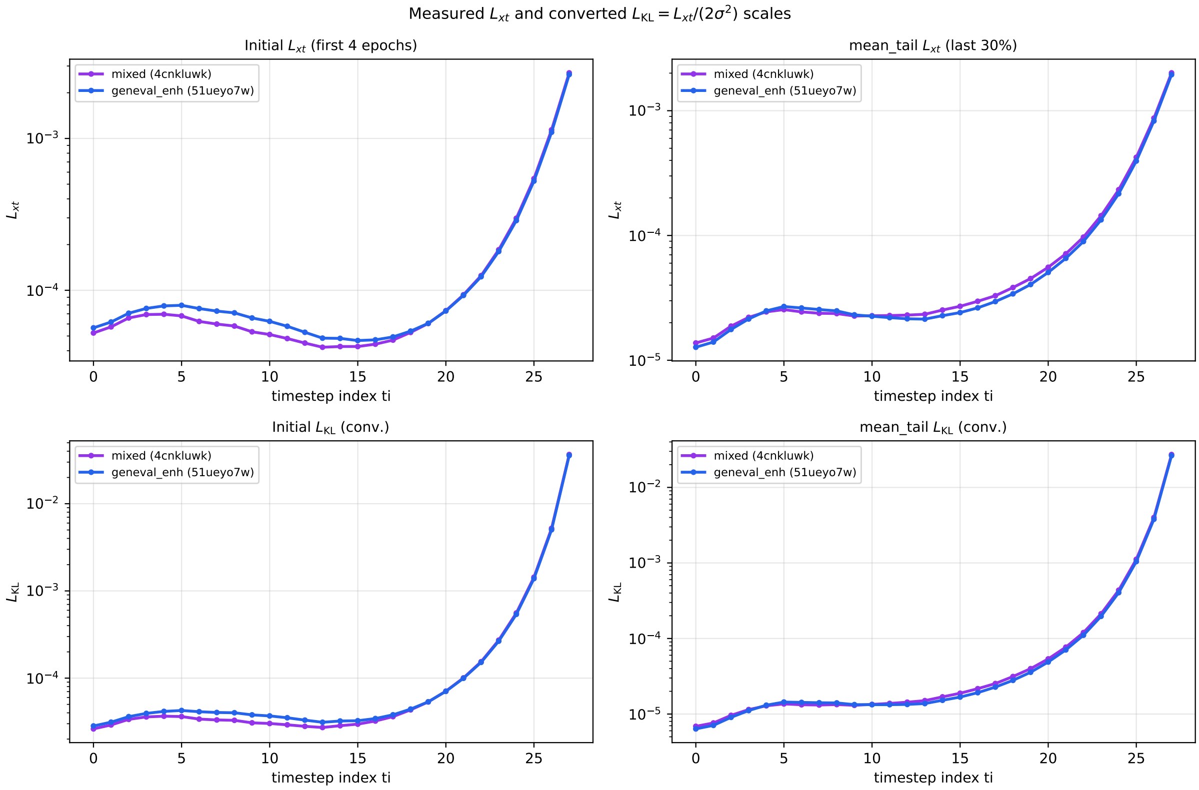

Figure 3: Top row: measured $L_{xt,k}$ (log-y); bottom row: converted $L_{\mathrm{KL},k} = L_{xt,k}/(2\sigma_k^2)$. Both runs show the L_{xt} U-shape; L_{KL} is even more concentrated on late `ti` because of the $1/\sigma_k^2$ factor.

46% on `ti=27` for logged L_{xt} — the curve matches *roughly constant velocity error* $\times \Delta t_k^2$, not a separate "mystery" effect.

- If instead $L_{x0,k} \equiv$ const: $L_{xt,k} = (\Delta t_k/\sigma_k)^2 L_{x0,k}$ would put **77**% on `ti=27`. The observed **50**$\times$ ratio $L_{\mu,27}/L_{\mu,13}$ is *between* the flat-L_v prediction ($\sim 148\times$ from Δt ratio alone, before L_v shrinkage) and the flat-L_{x0} prediction ($\sim 3100\times$), because both the metric *and* the actual L_v vary with k.
- Switching to L_{x_0} (`xopd_dk_space="x0"`) replaces the Δt_k^2 weighting with σ_k^2 (col. "%L_{x_0}"), up-weighting high-noise structure steps.

This parallels diffusion training, where ε-prediction MSE needs SNR reweighting to avoid low-noise dominance; here the analogue of "native" FM regression is L_v, not L_{xt}.

6.2 Cause B — genuine difficulty near the data manifold (a real signal)

Near clean, the posterior $\mathbb{E}[z_0 \mid x_t]$ sharpens, the velocity field's spatial Jacobian (Lipschitz constant) grows $\sim 1/\sigma$, and the target encodes sharp, high-frequency, multimodal detail. Under `normalize_d_k=false` the latent scale is also largest at clean. This part of the late-step cost is **real** and tied to final-image quality.

6.3 The U-shape

The observation doc's minimum at ti$\approx$ 12–13 and mild high-noise bump at ti$=0$ follow from $L_{xt} = \Delta t^2 L_v$ with L_v largest at high noise: early steps have small Δt^2 but large L_v; late steps have large Δt^2 but shrinking L_v; the minimum is where these cross.

Corollary: `normalize_d_k=true` makes it worse, not better

`normalize_d_k=true` divides by $2\bar{\sigma}^2 = 2\,\mathrm{std\_dev}_t^2\,|\Delta t|$, and $\bar{\sigma} \to 0$ toward clean, stacking a *second* amplification on top of $1/\sigma^2$. This is why transition-KL objectives (DanceGRPO/DDPO-style) intrinsically favor the low-noise region. **To equalize timesteps you must go the opposite way.**

7 XOPD-specific reading (large teacher)

"XOPD benefits more because the teacher is larger" is correct, but the benefit lives in **Cause B**, not A:

- The 32B→4B teacher's advantage (fine detail, texture, aesthetics) concentrates in the *low-noise* steps, where on-policy x_t is already near the student's final sample — the most valuable place for a big teacher to "polish".
- Structural/semantic correctness (geneval count/position, OCR) is decided at *high-noise* steps.

So "large teacher" does **not** imply "train only late steps". It implies "the late-step Cause-B component is worth keeping — but do not mistake the Cause-A amplification for that value."

8 Unified view: every proposal is a choice of per-step weight w_k

All proposals choose how to weight the underlying velocity gap $L_{v,k}$ across steps: $\mathcal{L} = \sum_k w_k L_{v,k}$ (equivalently $\sum_k w'_k L_{\{\mathrm{xt},v,x_0,\mathrm{KL}\},k}$).

direction	loss / weight on L_v	effect
default	L_{xt}, $w_k = 1 \Rightarrow w_k L_v = \Delta t_k^2$	late-step dominance (41% on ti=27 if L_v flat)
transition KL (theory)	L_{KL}, $w_k = \Delta t_k^2/(2\sigma_k^2)$	stronger late dominance than L_{xt} (77% on ti=27); DanceOPD trains plain L_v instead
1. late-only (DanceOPD)	L_{xt} or L_v on $\sigma_k < \tau$ only	polish; drops structure learning
2a. L_{x_0} (implemented)	L_{x_0}, $w_k = 1 \Rightarrow w_k L_v = \sigma_k^2$	up-weights high-noise steps
2b. L_v (implemented)	`xopd_dk_space="v"`	native FM velocity MSE
2c. EMA balance	$L_{xt}/\mathrm{EMA}(L_{xt})$	flattens logged scale; may suppress Cause B
3. endpoint-sensitivity	weight $\propto \partial z_0/\partial v_k$	2a is first-order proxy

8.1 Evaluation

- **Direction 1 (late-only)** trains L_{xt} on a late window only; same Δt_k^2 factor within the window. Drops high-noise steps where L_{x_0} (hence structure) matters most.
- **Direction 2a (L_{x_0})** replaces Δt_k^2 with σ_k^2 on the shared L_v. **Direction 2b (L_v)** removes the Δt_k^2 factor entirely (§4). **Do not use `normalize_d_k=true`** to equalize (§6.3).
- **Direction 3 (sensitivity)** idealizes weighting L_v by endpoint Jacobian; L_{x_0} is a good first-order proxy.

Recommendation

Compare `xopd_dk_space` $\in$ {`xt`, `v`, `x0`} on OCR (§9): default `xt` vs. velocity `v` vs. clean-latent `x0`, plus late-only `xt` (direction 1).

9 Disambiguating experiment (artifact vs. signal)

The ablation in `docs/xopd/progress/2026-07-12-ablation-relay.md` (all on OCR, otherwise identical) is exactly the test:

variant	**config**	**reads**
full-timestep xt-MSE	`flux2_klein_32b_to_4b_l1_ocr_1kep.yaml`	default (`xt`)
late-timestep xt-MSE	`flux2_klein_32b_to_4b_l1_ocr_selective_teacher_1kep.yaml`	direction 1
full-timestep x0-MSE	`flux2_klein_32b_to_4b_l1_ocr_xspace_1kep.yaml`	direction 2a
full-timestep v-MSE	`flux2_klein_32b_to_4b_l1_ocr_vmse_1kep.yaml`	direction 2b

Interpretation:

- If `x0` or `v` **improves** high-noise-half learning and structure metrics *without* degrading clean-end detail $\rightarrow$ the original `xt` dominance was mostly **Cause A (artifact)**; reweighting is a net win.
- If `x0`/`v` **hurts** clean-end detail with insufficient structure gains $\rightarrow$ the late-step **Cause B (real value)** dominates; consider soft late-window `xt`.

Remark. *Logged* `train/d_k/{ti}` *magnitudes differ across* `v`/`xt`/`x0`*; compare per-timestep* shape *and downstream eval (§4), not raw values across spaces.*

10 Adaptive self-normalized reweighting (DiffusionNFT / DMD) for the OPD d_k

Empirical motivation. With `xopd_dk_space`$\in$ {`v`,`x0`} (the two that win, and to within noise are equal) the per-step magnitude d_k is *tilted the other way* from the `xt` case of §4: it is **larger at early (high-noise) steps and smaller at late (clean) steps**. A single optimizer update over the 28-step trajectory is therefore dominated by the early steps, and the clean end (which carries fine detail) is under-trained. We ask whether a *per-step reweighting* that equalizes the step contributions helps.

DiffusionNFT's weighting. DiffusionNFT (Zheng et al., 2025, NVlabs/DiffusionNFT) replaces the manual $w(t)$ of Eq. (1) with a *self-normalized* $\boldsymbol{x}_0$ *regression* borrowed from DMD (Yin et al., 2024):

$$w(t)\ \|\boldsymbol{v}_\theta(\boldsymbol{x}_t,\boldsymbol{c},t)-\boldsymbol{v}\|_2^2 \longleftarrow \frac{\|\boldsymbol{x}_\theta(\boldsymbol{x}_t,\boldsymbol{c},t)-\boldsymbol{x}_0\|_2^2}{\mathrm{sg}\big(\mathrm{mean}\,|\,\boldsymbol{x}_\theta(\boldsymbol{x}_t,\boldsymbol{c},t)-\boldsymbol{x}_0\,|\big)}, \tag{17}$$

with $\boldsymbol{x}_\theta=\boldsymbol{x}_t-t\,\boldsymbol{v}_\theta$ (rectified flow) and sg the stop-gradient. The denominator is a *detached* per-sample scale (mean absolute $\boldsymbol{x}_0$ error), so (17) is a **scale-invariant** regression: its gradient is the ordinary $\boldsymbol{x}_0$-MSE gradient divided by its own error magnitude, giving every (sample, t) a gradient of comparable RMS.

Implied time weight. Under rectified flow the target is $\boldsymbol{x}_0 = \boldsymbol{x}_t - t\boldsymbol{v}$, so $\boldsymbol{x}_\theta - \boldsymbol{x}_0 = -t(\boldsymbol{v}_\theta - \boldsymbol{v})$ and $\mathrm{mean}|\boldsymbol{x}_\theta - \boldsymbol{x}_0| = t\,\mathrm{mean}|\boldsymbol{v}_\theta - \boldsymbol{v}|$. Substituting into (17),

$$(17) = \underbrace{\frac{t}{\mathrm{sg}\big(\mathrm{mean}\,|\boldsymbol{v}_\theta - \boldsymbol{v}|\big)}}_{=:\, w_{\mathrm{eff}}(t)} \|\boldsymbol{v}_\theta - \boldsymbol{v}\|_2^2 \,. \tag{18}$$

Two facts follow. (i) The scheme is **not** a naive equalizer: it keeps an explicit t factor that *up-weights high noise* — exactly the direction the paper reports is stable, whereas the inverse $w(t) = 1 - t$ (up-weighting the clean end) *collapses.* (ii) On top of the t tilt it divides by the *measured* per-step error scale, i.e. it adapts to the very early-large/late-small profile we observe and flattens the *effective gradient* without hard-flipping the tilt.

Transfer to the OPD d_k (teacher target). Our d_k is a *distillation* MSE whose target is the frozen teacher's prediction, not data. The construction transfers verbatim by replacing $\boldsymbol{x}_0$ with the teacher clean-latent $\boldsymbol{x}_0^{\mathrm{t}}$ and $\boldsymbol{x}_\theta$ with the student $\boldsymbol{x}_0^{\mathrm{s}}$:

$$d_k^{\mathrm{norm}} = L_{x_0}^{\mathrm{norm}} = \frac{\big\|\boldsymbol{x}_0^{\mathrm{s}} - \boldsymbol{x}_0^{\mathrm{t}}\big\|_2^2}{\mathrm{sg}\big(\mathrm{mean}\,|\,\boldsymbol{x}_0^{\mathrm{s}} - \boldsymbol{x}_0^{\mathrm{t}}\,|\big) + \varepsilon} = \frac{t}{\mathrm{sg}\big(\mathrm{mean}\,|\boldsymbol{v}_{\mathrm{s}} - \boldsymbol{v}_{\mathrm{t}}|\big) + \varepsilon} \|\boldsymbol{v}_{\mathrm{s}} - \boldsymbol{v}_{\mathrm{t}}\|_2^2 \,, \tag{19}$$

(definition (4); config `xopd_dk_space="x0_norm"`), applied independently at each trajectory step k (with $t = t_k$). The two forms are identical because $\boldsymbol{x}_0^{\mathrm{s}} - \boldsymbol{x}_0^{\mathrm{t}} = -t(\boldsymbol{v}_{\mathrm{s}} - \boldsymbol{v}_{\mathrm{t}})$; the second is the cheaper drop-in for a $\boldsymbol{v}$-space d_k. The floor $\varepsilon > 0$ guards the denominator (the teacher-student gap $\to 0$ near convergence, so $1/c_k \to \infty$ without it).

Why this may beat the fixed `normalize_d_k`. `normalize_d_k` divides d_k by $2\bar{\sigma}^2$ — a *fixed*, schedule-only reweighting equivalent to a particular t-profile. Eq. (19) instead divides by the *measured* per-step teacher-student error, so it auto-tunes to the empirical early-large/late-small shape rather than a prior. It is the data-adaptive analogue of the loss-space identity $\mathrm{MSE}(v) : \mathrm{MSE}(x_t) : \mathrm{MSE}(x_0) = 1 : \Delta t^2 : \sigma^2$: instead of choosing a space to reweight, it normalizes each step by its own realized magnitude.

Predictions and risks.

- **If** the early-step dominance was mostly a scale artifact, (19) should lift clean-end fidelity (detail-sensitive eval: OCR text, geneval attributes) at equal compute, mirroring the 25× efficiency DiffusionNFT reports.
- **Risk (collapse).** Do *not* implement a hard equalizer or inverse tilt ($w \propto 1-t$ or $w \propto 1/\|d_k\|$ that cancels the t factor): the paper shows this collapses. Keep the t factor of (18) (i.e. normalize in $\boldsymbol{x}_0$ space, not raw $\boldsymbol{v}$ space) so high noise stays up-weighted.
- **Stop-gradient is load-bearing**: without sg the denominator enters the loss surface and the objective degenerates.
- **Convergence**: as the teacher-student gap shrinks the effective weight grows; monitor for late-training instability and rely on ε + `max_grad_norm`.

Implementation sketch. Add `xopd_dk_weighting` $\in$ {`none`, `selfnorm_x0`}; when `selfnorm_x0`, compute the per-step $\boldsymbol{x}_0$ (or $\boldsymbol{v}$) student $-$ teacher error, divide the per-step d_k by $\mathrm{sg}(\mathrm{mean}|\cdot|) + \varepsilon$ per (19), and keep it orthogonal to `xopd_dk_space` (it *subsumes* the fixed `normalize_d_k`). Ablate against `none` on the v-MSE and x_0-MSE arms.

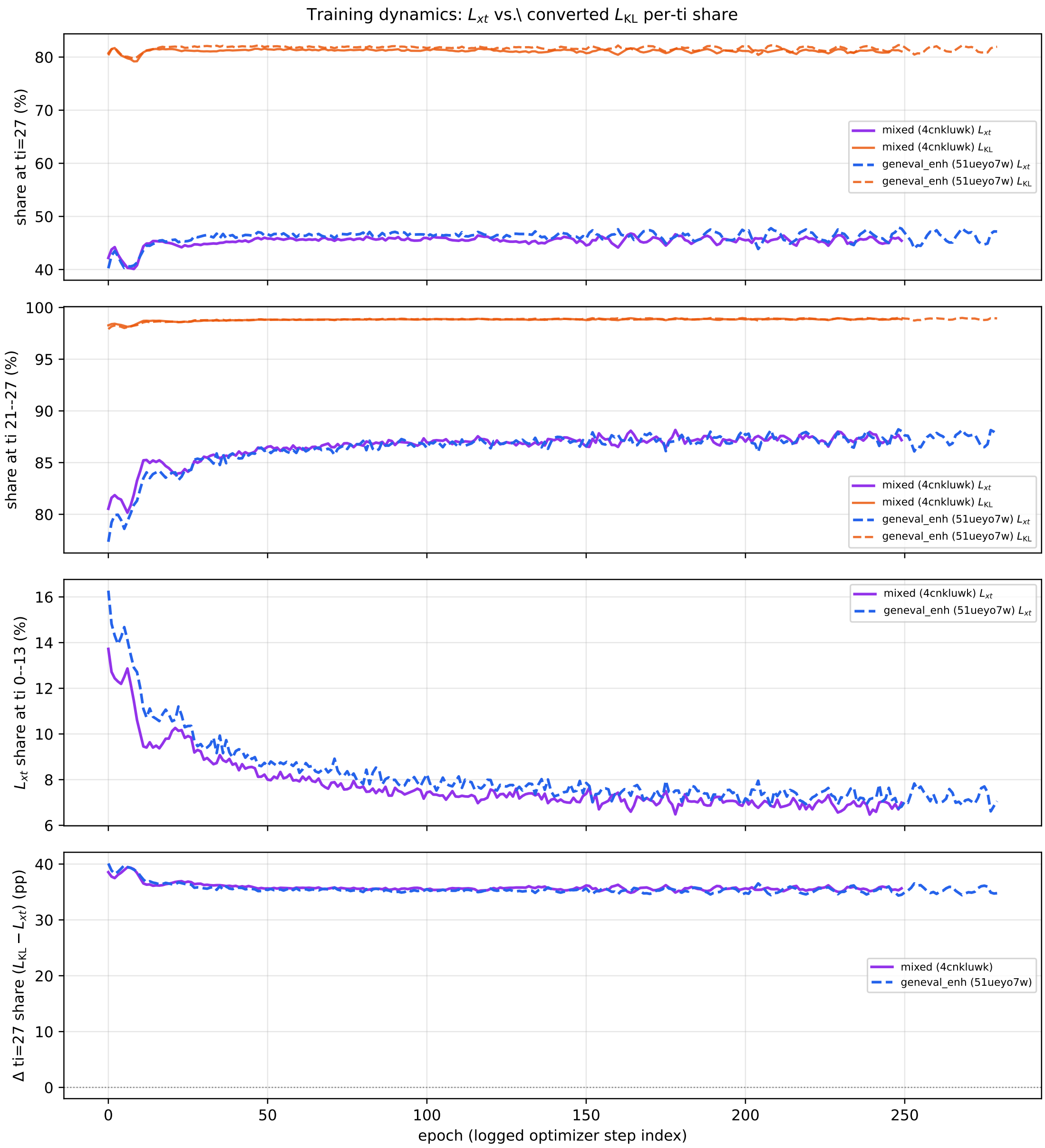

Figure 4: L_{xt} (run color) vs. converted L_{KL} (orange, same line style per run) per-timestep share vs. epoch. Mixed (`4cnkluwk`, solid) and geneval_enh (`51ueyo7w`, dashed). Bottom: Δ ti=27 share ($L_{\mathrm{KL}} - L_{xt}$).